

MAGEASY Replenishment & Advertising Dashboard

Multi-marketplace Amazon FBA inventory, sales, and advertising management system for US / UK / DE.

1. Project Overview

This system automates daily data ingestion from Amazon SP-API (inventory, sales, orders, fees, advertising) into a local DuckDB database, exposes it via a FastAPI service, and renders analytics in a single-page HTML dashboard. It computes per-SKU replenishment recommendations by combining Anchor (product master), Shenzhen warehouse (factory-side stock), and Amazon FBA / AWD inventory.

The system supports three Amazon marketplaces (US, UK, DE) with isolated per-site inventory and sales data, while sharing the product catalog and factory warehouse across all sites.

2. Architecture

2.1 Service containers (docker-compose.yml)

Container	Port	Description
sp_etl	8000	FastAPI service (etl/main.py) + static dashboard
sp_n8n	5678	n8n scheduler for automated workflows

2.2 Data layer

DuckDB (/data/sp_api.duckdb) — primary business data store.

Table	Purpose	Marketplace Scope
inventory	Amazon FBA inventory snapshots	Per-site (PK includes marketplace_id)
awd_inventory	Amazon Warehouse & Distribution stock	US only (no marketplace column)
sales_summary	Daily sales aggregates	Per-site
orders / order_items	Orders and line items	Per-site
sales_traffic	Session, Buy Box, CVR monthly reports	Per-site
ads_sponsored_products	Sponsored Products ad data	Currently US only
sz_warehouse	Shenzhen warehouse (factory-side stock)	Shared across sites
product_catalog	MAGEASY Anchor master (SKU-ASIN)	Shared across sites
sku_config	Product type & case pack settings	Shared
replenishment_controls	Dynamic replenishment thresholds	Shared
sync_anomalies	Data drift detection log	Per-site

SQLite (/data/sp_api_meta.db) — pipeline execution log (run_id / status / error).

3. Multi-marketplace Design

3.1 Configuration (.env)

```
SP_API_MARKETPLACE_IDS=["ATVPDKIKX0DER", "A1F83G8C2AR07P", "A1PA6795UKMFR9"]
SP_API_REFRESH_TOKEN=Atzr|... # NA region (US/CA/MX)
SP_API_REFRESH_TOKEN_EU=Atzr|... # EU region (UK/DE/FR/IT/ES)
SP_API_REFRESH_TOKEN_FE= # FE region (JP/AU/SG), unused
```

3.2 Marketplace IDs

Site	Marketplace ID	Region
US	ATVPDKIKX0DER	NA
UK	A1F83G8C2AR07P	EU
DE	A1PA6795UKMFR9	EU

3.3 Data isolation matrix

Data Type	US	UK	DE	Isolation Method
FBA Inventory	Independent	Independent	Independent	inventory.marketplace_id
AWD Inventory	Shown	0	0	AWD is US-only service
Sales	Independent	Independent	Independent	Split by currency (USD/GBP/EUR)
Orders	Independent	Independent	Independent	orders.marketplace_id
Shenzhen / JL warehouse	Shared	Shared	Shared	Factory-side stock
Product catalog (Anchor)	Shared	Shared	Shared	One master file
Fees / Traffic	Independent	Independent	Independent	Router loops all sites

3.4 Key architectural decisions

- **inventory PK includes marketplace_id** — prevents SP-API multi-site loop from overwriting each other's rows (same ASIN+SKU across sites).
- **Sales report is region-level** — one EU API call returns all EU sites' data. Rows are split into UK / DE using the currency field (GBP → UK, EUR → DE).
- **AWD is US-only** — the replenishment SQL guards with `WHERE ? = 'ATVPDKIKX0DER'`, forcing AWD values to 0 on non-US dashboards.
- **Fees / Traffic routers accept optional marketplace_id** — when omitted, the router iterates `SP_API_MARKETPLACE_IDS`.
- **Anomaly detector keyed by (sku, marketplace_id)** — prevents cross-site SKU collisions in the `sync_anomalies` log.

4. Core Business Logic

The system uses **ASIN as the primary business key** — not SKU. A single ASIN may map to multiple SKUs (e.g., version changes). The replenishment engine (`services/replenishment_service.py`) aggregates data across all SKUs sharing an ASIN.

4.1 Coverage formulas

```
Monthly Sales (H) = sales_30d
FBA Total (J) = fulfillable + fc_transfers + fc_processing
+ inbound_working + inbound_shipped + inbound_receiving
FBA Month (K) = FBA Total / Monthly Sales

AWD Total (O) = AWD Available + AWD Inbound - AWD Outbound
AWD Month (P) = AWD Total / Monthly Sales

Total Coverage (Q) = (FBA Total + AWD Total) / Monthly Sales
```

4.2 Virtual US inventory

```
US virtual stock = us_qty (on-hand) + pending_qty (ordered, not received)
US virtual Month = US virtual stock / Monthly Sales
```

4.3 Transfer & Reorder logic

```
safety_stock = monthly_sales * transfer_level (by product type)
Normal / Hero / Rocket (default 2.5)

us_gap = max(0, safety_stock - US virtual stock)

transfer_qty = min(JL warehouse stock, us_gap)
Suppress: if Total Coverage >= trigger (default 4.0) -> transfer_qty = 0

reorder_qty = max(0, us_gap - JL warehouse stock)
Suppress: if Total Coverage >= trigger (default 4.0) -> reorder_qty = 0

AWD replenishment:
need = awd_target * monthly_sales - AWD Total
cap = 3.0 * monthly_sales - (FBA Total + AWD Total)
awd_replen = min(need, cap)
```

4.4 Three-tier alert system (priority: red > yellow > blue)

Alert	Condition
Red — Reorder Needed	<code>reorder_qty > 0</code>
Yellow — JL Warehouse Critical	<code>transfer_qty > 0 AND (JL - transfer) < safety_stock * 0.5</code>
Blue — Ordered, Awaiting Receipt	<code>pending_qty > 0 AND reorder_qty = 0</code>

4.5 Product types (`sku_config.product_type`)

- **Normal** — default replenishment cadence
- **Hero** — high-traffic products, higher stock levels
- **Rocket** — top priority, aggressive stocking (2.5 month default)

- **Discontinued** — no reorder / transfer / AWD replenishment triggered

5. Main API Endpoints

5.1 Inventory (FBA)

Method	Path	Purpose
POST	/etl/inventory/sync	Sync all marketplaces via SP-API
POST	/etl/inventory/upload?marketplace_id=US	Manual CSV upload (accepts form field too)
POST	/etl/inventory/manual-adjust	Single SKU manual correction
POST	/etl/inventory/remap-marketplace	Move rows between marketplaces
GET	/etl/inventory/debug/{asin_or_sku}	Diagnostic query

5.2 AWD (US only)

Method	Path	Purpose
POST	/etl/awd/sync	AWD inventory API sync
POST	/etl/awd/upload	Manual AWD Excel upload
POST	/etl/awd/report-sync	SP-API AWD Report + reconciliation

5.3 Sales

Method	Path	Purpose
POST	/etl/sales/sync	Loop all regions (no marketplace_id) or single site
GET	/etl/sales/marketplace-breakdown	Diagnostic: per-site data distribution
POST	/etl/sales/purge?marketplace_id=X	Clean bad data for a site

5.4 Replenishment (dashboard main data)

Method	Path	Purpose
GET	/replenishment?marketplace_id=US	Main data feed for dashboard
GET	/replenishment/daily-alert	Daily email alert data (called by n8n)

5.5 Health & Anomalies

Method	Path	Purpose
GET	/etl/health/summary?marketplace_id=US	Per-pipeline sync health
GET	/etl/health/anomalies?marketplace_id=US	Recent data anomalies list
POST	/etl/health/anomalies/ack	Acknowledge anomalies

6. Dashboard

The dashboard is a single-file HTML application (`etl/static/dashboard.html`) served at `http://localhost:8000/dashboard`.

Main panels

- **Marketplace filter dropdown** (US / UK / DE) — switching reloads data and health panels
- **Upload tools** — Anchor / SZ warehouse / AWD / FBA CSV; FBA upload has an independent red-bordered marketplace selector (decoupled from filter) with a confirmation dialog to prevent uploading to the wrong site
- **Data sync health panel** — pipeline status, freshness, coverage, anomalies
- **Anomaly panel** — data drift alerts from the past 24 hours
- **Replenishment master table** — one row per SKU with reorder / transfer / AWD suggestions and multi-tier alerts
- **Controls modal** (floating gear button) — adjust replenishment thresholds; changes persist to the `replenishment_controls` table

7. n8n Workflows (n8n/*.json)

Workflow	Cron	Purpose
awd_replenishment_workflow	0 10 * * * (10:00 UTC = 18:00 Taiwan)	Amazon ETL: FBA + AWD + Orders + Sales + Alert
workflow_orders	Every 4 hours	Orders sync
workflow_finance	0 23 * * *	Finance sync
workflow_ads	0 0 * * *	Ads sync (currently US only)
workflow_fees	0 4 2 * *	Monthly Fees sync (2nd of month)
workflow_traffic	0 3 2 * *	Monthly Traffic sync
workflow_traffic_mtd	0 9 * * *	Daily month-to-date Traffic
workflow_awd_report	0 3 * * *	AWD authoritative report reconcile

Important: Most n8n workflow HTTP nodes should not hard-code marketplace_id. The backend auto-loops all configured marketplaces.

Amazon 24h rate limit: Each (report type × marketplace) can only succeed once per 24-hour window. If sync succeeded at 10:00 UTC today, the next successful run must wait until after 10:00 UTC tomorrow.

8. Common Operations

Start / stop

```
docker compose up -d # Start
docker compose down # Stop (data preserved)
docker compose restart etl # Restart API (does NOT reload .env)
docker compose down && docker compose up -d # Full restart (reloads .env)
```

View logs

```
docker compose logs -f etl
docker compose logs --tail 100 etl | grep -i error
```

Read-only DB query (avoids FastAPI lock)

```
docker compose exec etl python3 -c "
import duckdb
c = duckdb.connect('/data/sp_api.duckdb', read_only=True)
# ... your query ...
"
```

9. Deployment to GCP

Checklist

- Compute Engine (recommended over Cloud Run — DuckDB needs persistent volume)
- Move .env to Google Secret Manager
- Mount Persistent Disk at /data for DuckDB file
- Deploy n8n separately (Docker or GCP-hosted)
- Open outbound firewall to sellingpartnerapi-*.amazon.com

- After deploy: run DB migration verification (check startup logs for db.migration_applied)

10. Architectural Decisions Log

- **inventory PK adds marketplace_id** — prevents multi-site SP-API loop from overwriting rows
- **FBA CSV upload accepts marketplace_id via query OR form field** — safety net for accidental site mismatch
- **Sales split by currency field** — Amazon Sales Report has no sales-channel column
- **AWD gated by marketplace check** — CTE only executes for US marketplace
- **Health & Anomaly APIs accept marketplace_id filter** — dashboard passes current site
- **Fees / Traffic routers loop all marketplaces** — parity with FBA/Sales/Orders
- **Sales sync grouped by region** — one call per region (NA, EU), currency split downstream

11. Pending Work

- **P1-5:** Add marketplace_id column to ads_sponsored_products (needed before running UK/DE ads)
- **P1-9:** Add marketplace loop to Ads n8n workflow (pair with P1-5)

12. Data Flow Overview

```

Amazon SP-API ---+
Amazon Ads API ---+----> n8n (schedule) ----> FastAPI (/etl/*/sync) ----> DuckDB
Manual upload ---+ |
v
FastAPI (/replenishment)
|
v
Dashboard HTML (single page)
|
(user tweaks Controls)
|
v
replenishment_controls table
(feeds next computation)

```

13. Repository Structure

```
etl/  
main.py # FastAPI application  
core/  
config.py # Settings (region_for, refresh_token_for)  
database.py # DuckDB schema + migrations  
sp_api_client.py # Region-aware SP-API client  
db_meta.py # SQLite pipeline runs  
routers/  
inventory.py # FBA endpoints  
sales.py / orders.py / fees.py / traffic.py / ads.py  
awd_upload.py # AWD Excel upload  
sz_warehouse.py # Shenzhen warehouse template  
catalog.py # MAGEASY Anchor upload  
replenishment.py # Dashboard main data  
health.py # Health / anomalies  
controls.py # Replenishment thresholds  
services/  
inventory_service.py # FBA sync  
awd_service.py # AWD sync  
awd_report_service.py # AWD authoritative report + reconcile  
sales_service.py # Region-level sync + currency split  
orders_service.py / fees_service.py / traffic_service.py / ads_service.py  
replenishment_service.py # Main computation (big CTE query)  
anomaly_detector.py # Data drift detection  
base_service.py # BaseService (start_run / finish_run)  
static/  
dashboard.html # Single-page dashboard  
  
n8n/ # n8n workflow JSON files  
.env # Environment (do NOT commit)  
docker-compose.yml
```